# Crime Rate Prediction Using Machine Learning

CS-UY 4563 - Introduction to Machine Learning | Final Project

Dataset: UCI Communities and Crime | Task: Regression

Tyler Huynh and Martha McQuillan

# A. Introduction

**Dataset.** This project uses the UCI Communities and Crime dataset, sourced from Kaggle. It contains socio-economic, demographic, and law enforcement data for over 2,000 communities across the United States, with roughly 120 features per community. These features cover things like household income, poverty rates, unemployment, education levels, population age groups, and police staffing. The value we are trying to predict is *ViolentCrimesPerPop*, which is the number of violent crimes per 100,000 people, already scaled to a range of 0 to 1.

**Prediction task.** Given the socio-economic and demographic profile of a community, our goal is to predict its violent crime rate. This is a regression problem. It is worth studying because accurate predictions can help identify which communities are at higher risk, guide decisions about where to allocate public safety resources, and show how poverty and inequality connect to crime.

# B. Exploratory and Unsupervised Analysis

**Preprocessing.** The raw dataset had 2,215 rows and 128 columns. We started by removing the first five columns, which are just identifiers like state name and community code and carry no predictive information. We then dropped any feature column where more than 50% of the values were missing, since there is not enough data in those columns to impute reliably. After that, we removed the small number of rows where the target value itself was missing. For the remaining missing values in other columns, we used median imputation, calculated only from the training set so that information from the validation and test sets does not leak into the model. We applied standard scaling the same way, fitting only on the training data. Finally, we split the data into 70% training, 15% validation, and 15% test before doing any of the above steps, to make sure the splits are clean.

**Target distribution.** The crime rate variable is heavily right-skewed: most communities cluster near 0.0 to 0.2 on the scale, but a small number have rates close to 1.0, as shown in Figure 1. To deal with this, we applied a log1p transformation to the target (that is, $\log(1 + y)$), which compresses the long right tail and produces a more symmetric distribution (Figure 2). This helps all three of our regression models fit the data more evenly instead of being pulled toward extreme outliers.

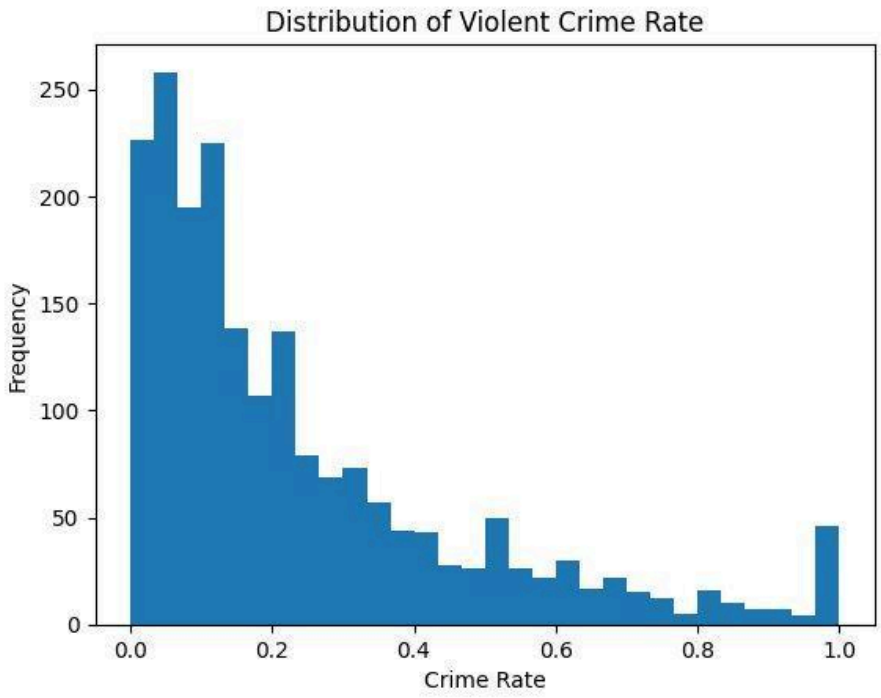


Figure 1. Distribution of ViolentCrimesPerPop across all communities

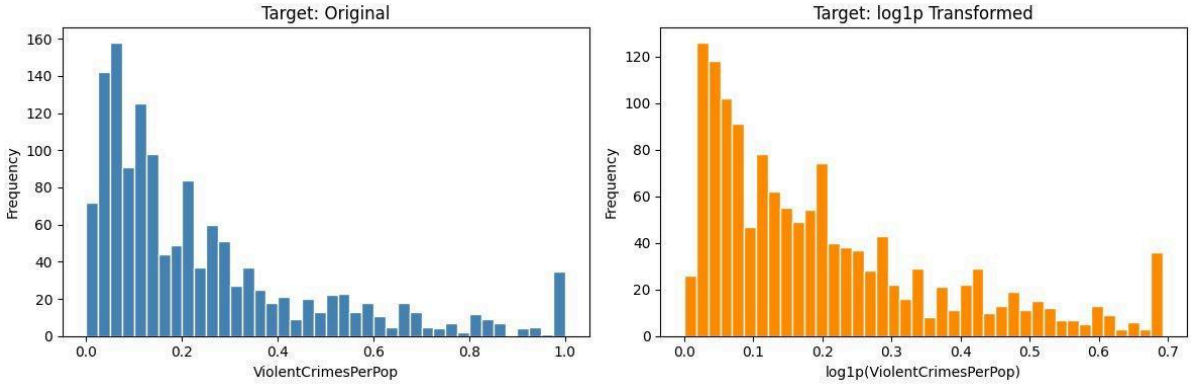


Figure 2. Target distribution before (left) and after (right) log1p transformation

**Missing values.** Figure 3 shows the distribution of missing data across all feature columns. The vast majority of features had complete or nearly complete data. However, a distinct cluster of approximately 22 columns had over 80% of their values missing. These were removed entirely during preprocessing, as imputing columns with so little real data would introduce more noise than signal.

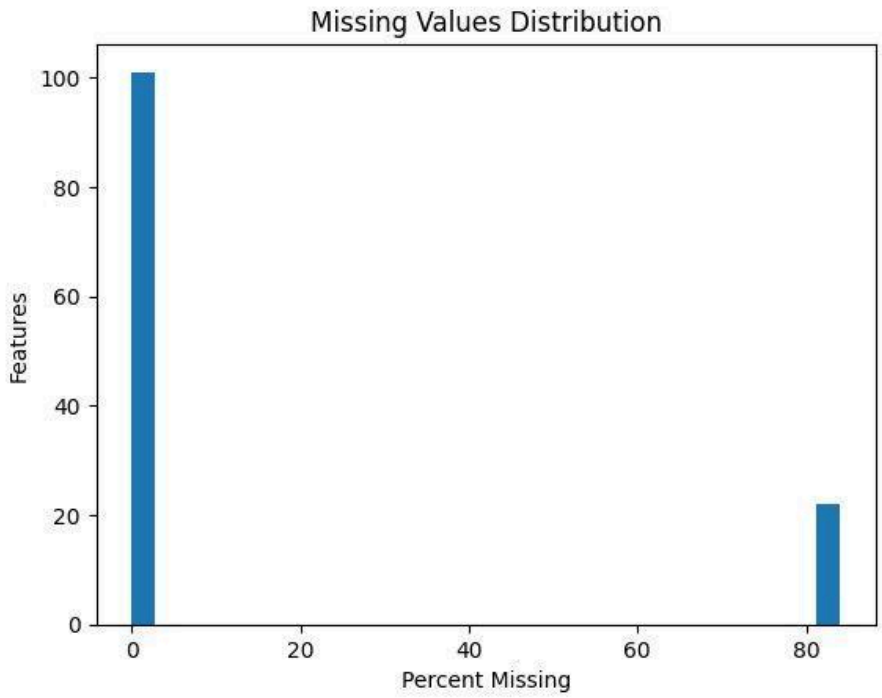


Figure 3. Distribution of missing values across all features

**Feature relationships.** Figures 4, 5, and 6 show scatter plots for three features that had among the strongest correlations with the crime rate. Features 48 and 49 both show a clear negative relationship: communities with higher values on these features tend to have lower crime rates. Feature 55 shows the opposite, it shows a positive relationship where higher values are associated with higher crime rates. These features likely represent income or wealth indicators (48 and 49) and a poverty or deprivation indicator (55). Importantly, all three scatter plots show curved rather than perfectly straight-line relationships, which is the key motivation for testing polynomial feature transformations in the modeling phase.

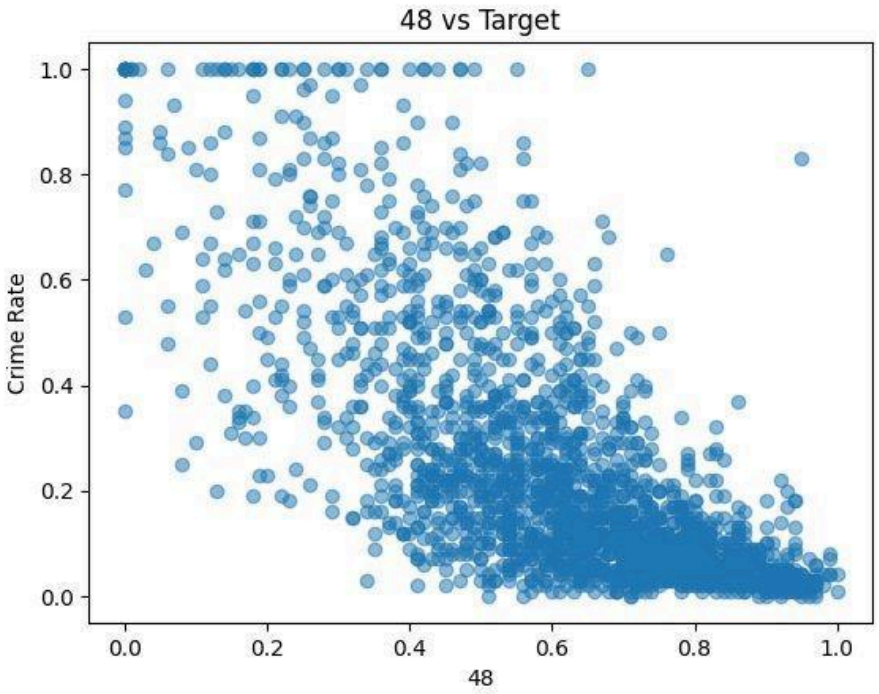


Figure 4. Feature 48 vs. violent crime rate

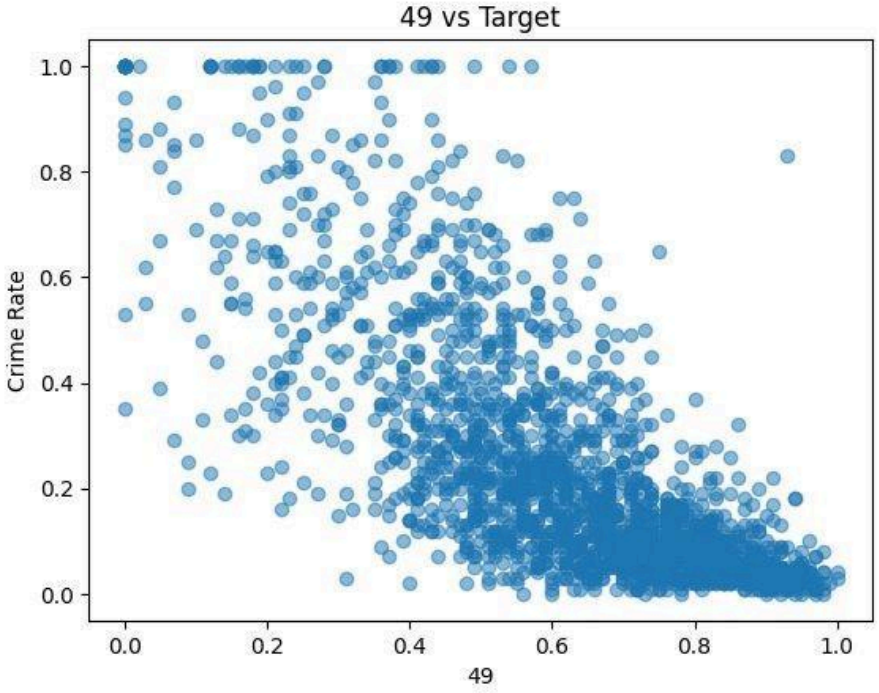


Figure 5. Feature 49 vs. violent crime rate

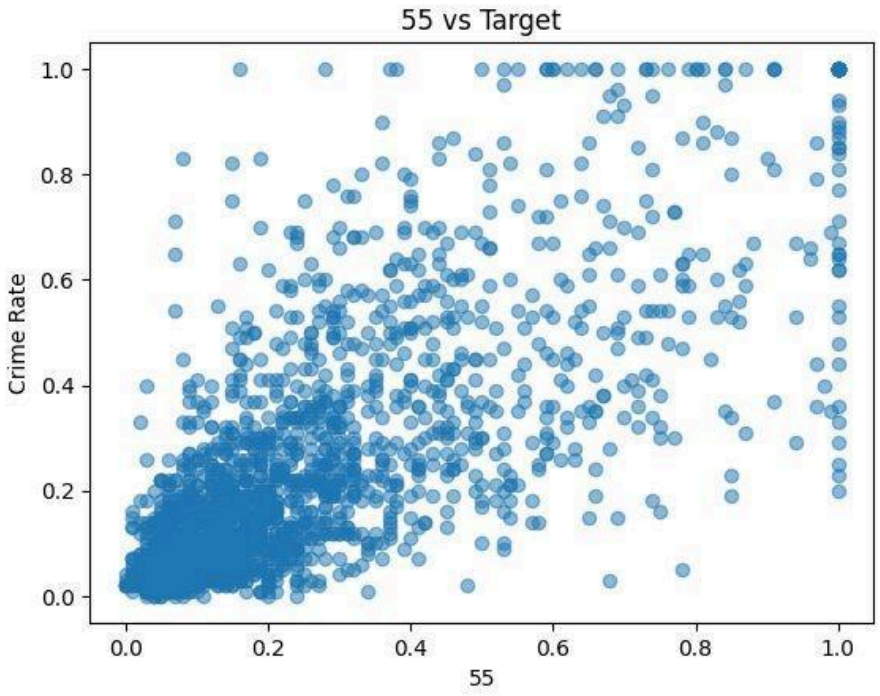


Figure 6. Feature 55 vs. violent crime rate

**Multicollinearity.** Figure 7 shows a correlation heatmap of the 15 features most strongly correlated with the target. Several features are highly correlated with each other. For example, Features 49, 48, 50, and 51 all show pairwise correlations above 0.90, and Features 45 and 46 are correlated at 0.98. This degree of multicollinearity means Ridge regression may struggle to isolate the individual contribution of each feature, since many of them carry nearly identical information. This is the primary motivation for using PCA as one of our feature transformations, as PCA replaces these correlated features with orthogonal components that do not overlap in the information they carry.

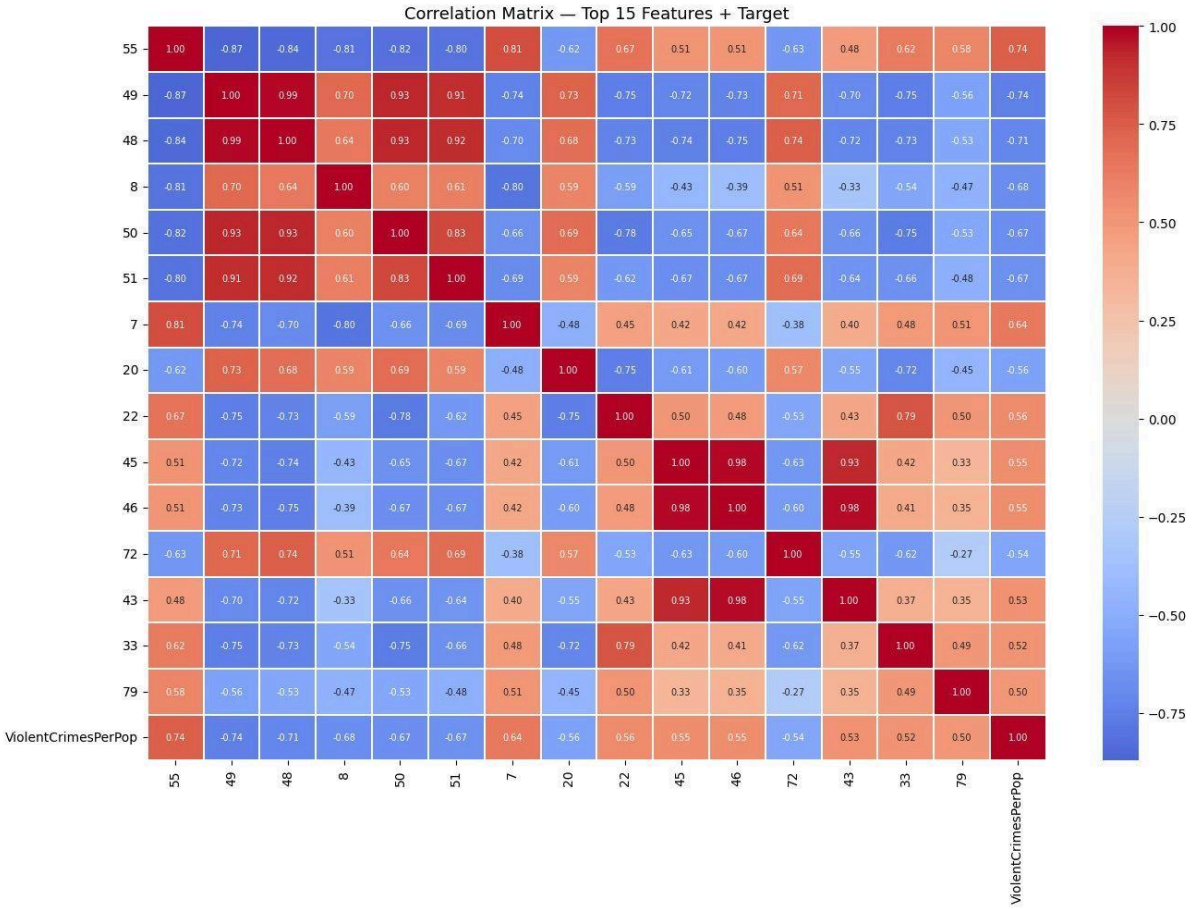


Figure 7. Correlation matrix of the 15 features most correlated with ViolentCrimesPerPop, plus the target. Red indicates positive correlation, blue indicates negative.

# C. Supervised Modeling

We trained three different model types: Ridge Regression, a Neural Network, and K-Nearest Neighbors (KNN). For each model, we tested four feature spaces: the original scaled features with no transformation, and three transformed versions. Within each feature space, we tried six different regularization settings. This gives us at least 24 models per approach and 72 models in total across all three approaches. All models were trained on the log-transformed target, and the test set was held out completely until final evaluation.

## Approach 1: Ridge Regression

**No transformation (baseline).** We first ran Ridge directly on the scaled original features. This gives us a performance floor to compare everything else against.

**Transformation 1 - Polynomial degree 2.** Because the scatter plots in Figures 4–6 showed curved relationships between several features and the crime rate, we created degree-2 polynomial features. This means we added squared versions of each feature ($x^2$) and all pairwise products between features ($x_i \times x_j$). This lets a linear model like Ridge fit curved patterns without changing the model itself. After this expansion, we re-scaled the features since the polynomial terms have very different magnitudes than the originals.

**Transformation 2 - PCA(20) then polynomial degree 3 (chained).** Applying degree-3 directly to the full set of features would produce an unmanageable number of terms, so we first reduced the feature space to 20 principal components using PCA, then applied the degree-3 polynomial to those 20 components. This chained step counts as one transformation. It is the most complex of the three Ridge transformations and is expected to need strong regularization to avoid overfitting.

**Transformation 3 - PCA (95% variance).** We applied PCA and kept enough components to retain 95% of the total variance. This removes the multicollinearity problem visible in Figure 7 by replacing the original correlated features with a smaller set of independent components. We expected this transformation to generalize better than the polynomial ones because it reduces noise rather than adding complexity.

**Regularization.** Ridge regression controls overfitting through the lambda (λ) parameter, which shrinks the model's coefficients toward zero. A larger lambda means more shrinkage and a simpler model. We tested six values: 0.001, 0.01, 0.1, 1, 10, and 100.

## Approach 2: Neural Network

**Architecture.** We built a fully-connected neural network in PyTorch. The baseline version has one hidden layer with 128 units, as required for the neural network baseline. For all three transformations, the network uses two hidden layers. Each hidden layer applies batch normalization, ReLU activation, and dropout at a rate of 0.2 to reduce overfitting. We trained with the Adam optimizer and used early stopping with a patience of 20 epochs.

**No transformation (baseline).** One hidden layer (128 units) applied to the scaled original features.

**Transformation 1 - Polynomial degree 2.** Same reasoning as Ridge: the curved patterns in Figures 4–6 suggest the relationship is nonlinear. The polynomial expansion gives the first hidden layer a richer set of linear combinations to work with before the nonlinear activations are applied.

**Transformation 2 - PCA (95% variance).** PCA reduces the input to a smaller set of independent components, which shrinks the input layer and removes redundant signal. Feeding in highly correlated inputs as seen in Figure 7 can make gradient-based learning unstable, so decorrelating the features first is expected to help.

**Transformation 3 - K-Means cluster distances (k=20).** We fit a K-Means model with 20 clusters on the training data, then replaced each community's feature vector with its distance to each of the 20 cluster centers. This gives the network a 20-dimensional representation that reflects which type of community a data point is closest to. Communities naturally group by socio-economic profile, so this transformation is designed to give the network a compact representation that already encodes some of that structure.

**Regularization.** We used the Adam optimizer's weight decay parameter as our form of L2 regularization. We tested six values: 0.0, 1e-4, 1e-3, 1e-2, 1e-1, and 1.0. A value of 0.0 means no regularization at all, which serves as a reference point for how much the model overfits without any constraint.

## Approach 3: K-Nearest Neighbors (KNN)

For our third model we used KNN, used as a regression model as opposed to a classification one, where each prediction is the average of the k nearest neighbors in the training set. Unlike Ridge and the Neural Network, KNN has no explicit regularization parameter. Instead, the number of neighbors k controls the complexity.

**No transformation - baseline.**

**Transformation 1 - Polynomial degree 2.** Figures 4-6 suggest a curved or nonlinear relationship, thus the degree 2 polynomial transformation was used. A polynomial model can help KNN see nonlinear or curved relationships if they do exist.

**Transformation 2 - PCA 95% variance.** The dataset contains many correlated features (figure 7), which can make distance based methods like KNN less reliable. PCA was applied to reduce the feature space while retaining 95% of the total variance. The transformation reduces redundancy and noise, and produces a lower dimension representation.

**Transformation 3 - k-means cluster (k=20).** Communities may naturally group into clusters based on similar socioeconomic or demographic characteristics. K-means with k=20 was used to capture this structure. This transformation helps KNN by providing a more structured representation of the data.

## D. Table of Results

The table below shows the best result achieved per model and feature space, chosen by lowest validation MSE across the six regularization values tested. All MSE values use the log1p-transformed target. The full table including all 72+ individual models and every regularization value is in the executed notebook.

| Feature Space | λ | Train MSE | Val MSE | Val $R^2$ | Observation |
|---|---|---|---|---|---|
| No Transformation | 0.001 | 0.00841 | 0.00866 | 0.677 | Tiny gap (0.000247) well-balanced, no overfitting |
| No Transformation | 0.01 | 0.00841 | 0.00866 | 0.677 | Identical to λ=0.001 regularization not yet affecting model |
| No Transformation | 0.1 | 0.00841 | 0.00866 | 0.677 | Still negligible change baseline features do not overfit |
| No Transformation | 1 | 0.00842 | 0.00866 | 0.677 | Train MSE begins rising slightly mild shrinkage starting |
| No Transformation | 10 | 0.00847 | 0.00866 | 0.677 | Train MSE rising further; val MSE stable; slight underfitting |
| No Transformation | 100 | 0.00879 | 0.00849 | 0.684 | Best val MSE here; negative gap means mild underfitting but val improved |
| Polynomial Degree 2 | 0.001 | 0.00000 | 0.05329 | −0.986 | Train MSE ≈ 0 severe overfitting; model memorized training data |
| Polynomial Degree 2 | 0.01 | 0.00000 | 0.05325 | −0.985 | Still massively overfit; val MSE barely changed |
| Polynomial Degree 2 | 0.1 | 0.00000 | 0.05288 | −0.971 | Overfitting persists; negative $R^2$ means worse than predicting mean |

| Feature Space | λ | Train MSE | Val MSE | Val $R^2$ | Observation |
|---|---|---|---|---|---|
| Polynomial Degree 2 | 1 | 0.00001 | 0.04962 | −0.849 | Val MSE improving but still badly overfit; need far more regularization |
| Polynomial Degree 2 | 10 | 0.00025 | 0.03488 | −0.300 | Large improvement; gap narrowing<br>regularization finally helping |
| Polynomial Degree 2 | 100 | 0.00178 | 0.01707 | 0.364 | Best result for Poly-2; still well below baseline $R^2$ of 0.684 |
| PCA(20) + Poly Deg 3 | 0.001 | 0.00000 | 0.19366 | −6.218 | Extreme overfitting; highest complexity transformation behaves worst unregularized |
| PCA(20) + Poly Deg 3 | 0.01 | 0.00000 | 0.19166 | −6.143 | Negligible improvement; model still fully memorizing training data |
| PCA(20) + Poly Deg 3 | 0.1 | 0.00000 | 0.17545 | −5.539 | Val MSE declining; regularization beginning to constrain cubic terms |
| PCA(20) + Poly Deg 3 | 1 | 0.00009 | 0.11710 | −3.364 | Significant drop; confirms this space needs much stronger regularization than baseline |
| PCA(20) + Poly Deg 3 | 10 | 0.00068 | 0.04768 | −0.777 | Still overfit but approaching reasonable territory |
| PCA(20) + Poly Deg 3 | 100 | 0.00235 | 0.01742 | 0.351 | Best result; comparable to Poly-2 but still below baseline<br>cubic terms did not help |
| PCA (95% variance) | 0.001 | 0.00958 | 0.00842 | 0.686 | Negative gap<br>val MSE slightly below train; PCA generalized well immediately |
| PCA (95% variance) | 0.01 | 0.00958 | 0.00842 | 0.686 | Identical result<br>PCA feature space insensitive to small λ changes |
| PCA (95% variance) | 0.1 | 0.00958 | 0.00842 | 0.686 | Still stable<br>decorrelated features reduce need for heavy regularization |
| PCA (95% variance) | 1 | 0.00958 | 0.00842 | 0.686 | No meaningful change<br>confirms PCA handles multicollinearity better than raw features |
| PCA (95% variance) | 10 | 0.00958 | 0.00841 | 0.687 | Val MSE improving marginally with more shrinkage |
| PCA (95% variance) | 100 | 0.00959 | 0.00834 | 0.689 | Best val MSE and $R^2$ overall for Ridge |

| Feature Space | λ | Train MSE | Val MSE | Val R² | Observation |
|---|---|---|---|---|---|
| | | | | | PCA outperforms all other transformations |

Table 1: Ridge Regression results.

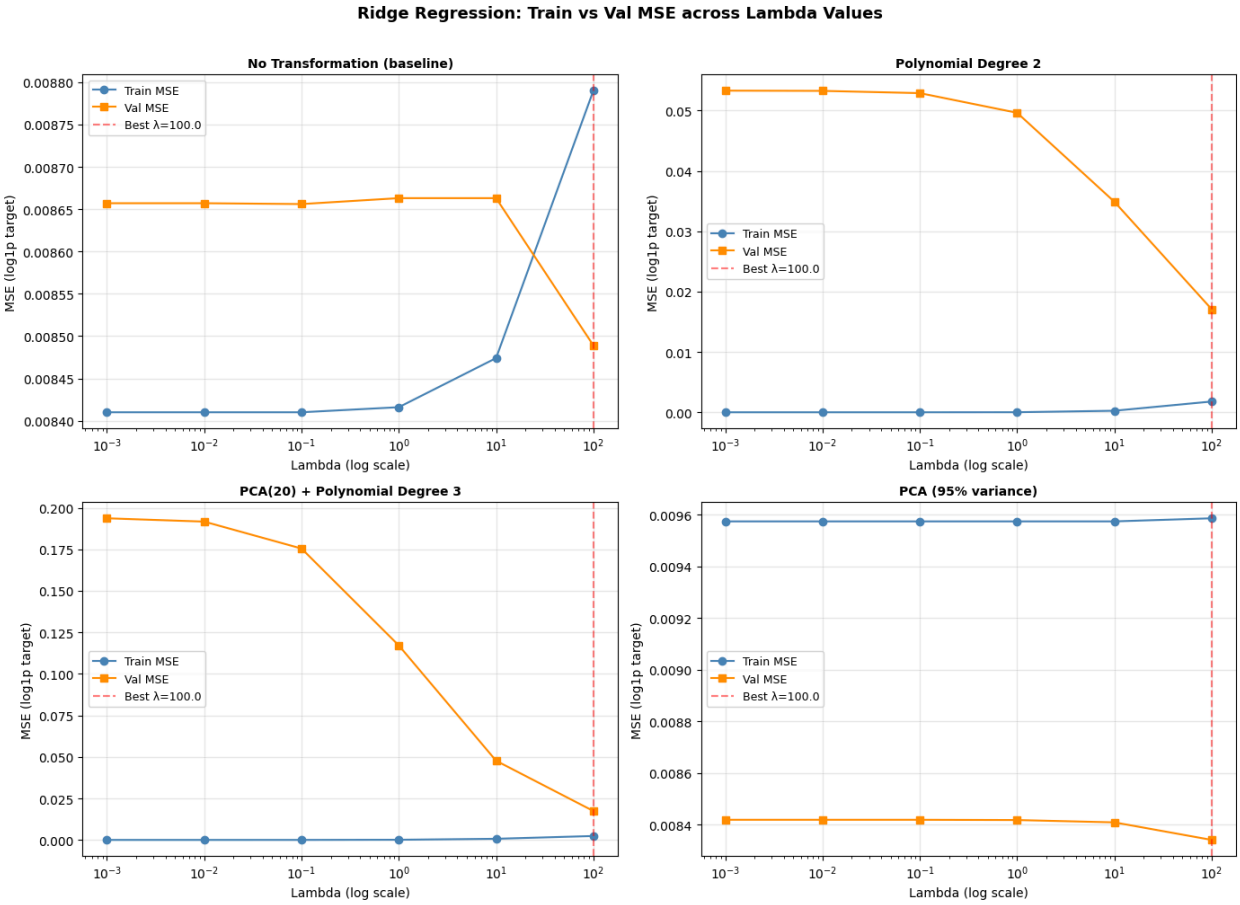


Figure 8: Ridge Regression: training vs. validation MSE across lambda values for all four feature spaces.

| Feature Space | Weight Decay | Train MSE | Val MSE | Val R² | Observation |
|---|---|---|---|---|---|
| No Transformation | 0.0 | 0.00681 | 0.00871 | 0.675 | Small gap (0.00191) mild overfitting without regularization |
| No Transformation | 1e-4 | 0.00680 | 0.00868 | 0.677 | Gap narrowing slightly weight decay beginning to help |
| No Transformation | 1e-3 | 0.00688 | 0.00860 | 0.680 | Val MSE improving; train rising slightly good regularization direction |
| No Transformation | 1e-2 | 0.00681 | 0.00863 | 0.678 | Marginal change; similar to 1e-3 |

| Feature Space | Weight Decay | Train MSE | Val MSE | Val $R^2$ | Observation |
|---|---|---|---|---|---|
| No Transformation | 0.1 | 0.00832 | 0.00855 | 0.681 | Best val MSE; gap near zero well-balanced bias/variance |
| No Transformation | 1.0 | 0.01081 | 0.00907 | 0.662 | Val MSE rising too much regularization causing underfitting |
| Polynomial Degree 2 | 0.0 | 0.00454 | 0.00939 | 0.650 | Gap of 0.00485 overfitting; expanded feature space needs regularization |
| Polynomial Degree 2 | 1e-4 | 0.00357 | 0.00924 | 0.656 | Train MSE dropped further slightly more overfit, not helping yet |
| Polynomial Degree 2 | 1e-3 | 0.00675 | 0.00927 | 0.655 | Gap narrowing but val MSE still above baseline transformation not improving results |
| Polynomial Degree 2 | 1e-2 | 0.00547 | 0.00920 | 0.657 | Small improvement; gap still present |
| Polynomial Degree 2 | 0.1 | 0.00633 | 0.00908 | 0.662 | Best for Poly-2; still underperforms baseline polynomial expansion did not help NN |
| Polynomial Degree 2 | 1.0 | 0.01004 | 0.01020 | 0.620 | Both MSEs rising over-regularized; model underfitting |
| PCA (95% variance) | 0.0 | 0.00777 | 0.00841 | 0.687 | Tiny gap (0.00064) PCA decorrelation helps generalization immediately |
| PCA (95% variance) | 1e-4 | 0.00775 | 0.00837 | 0.688 | Marginal improvement; gap remains small |
| PCA (95% variance) | 1e-3 | 0.00770 | 0.00826 | 0.692 | Best val MSE and $R^2$ for NN overall optimal regularization point |
| PCA (95% variance) | 1e-2 | 0.00718 | 0.00829 | 0.691 | Slight val MSE increase; still strong performance |
| PCA (95% variance) | 0.1 | 0.00898 | 0.00865 | 0.678 | Val MSE rising beginning to underfit |
| PCA (95% variance) | 1.0 | 0.00906 | 0.00872 | 0.675 | Over-regularized; both MSEs elevated |
| K-Means Dist. (k=20) | 0.0 | 0.00936 | 0.00871 | 0.675 | Negative gap val below train; compact feature space generalizes well |
| K-Means Dist. (k=20) | 1e-4 | 0.00935 | 0.00871 | 0.675 | Essentially unchanged |

| Feature Space | Weight Decay | Train MSE | Val MSE | Val $R^2$ | Observation |
|---|---|---|---|---|---|
| | | | | | feature space is low-dimensional and stable |
| K-Means Dist. (k=20) | 1e-3 | 0.00902 | 0.00841 | 0.687 | Best for K-Means; val MSE dropped<br>small weight decay beneficial here |
| K-Means Dist. (k=20) | 1e-2 | 0.00954 | 0.00859 | 0.680 | Val MSE rises slightly; 1e-3 was the sweet spot |
| K-Means Dist. (k=20) | 0.1 | 0.00936 | 0.00855 | 0.682 | Still reasonable but below best |
| K-Means Dist. (k=20) | 1.0 | 0.01236 | 0.01017 | 0.621 | Over-regularized; both MSEs spike<br>weight decay of 1.0 too strong for 20-dim space |

Table 2: Neural Network results.

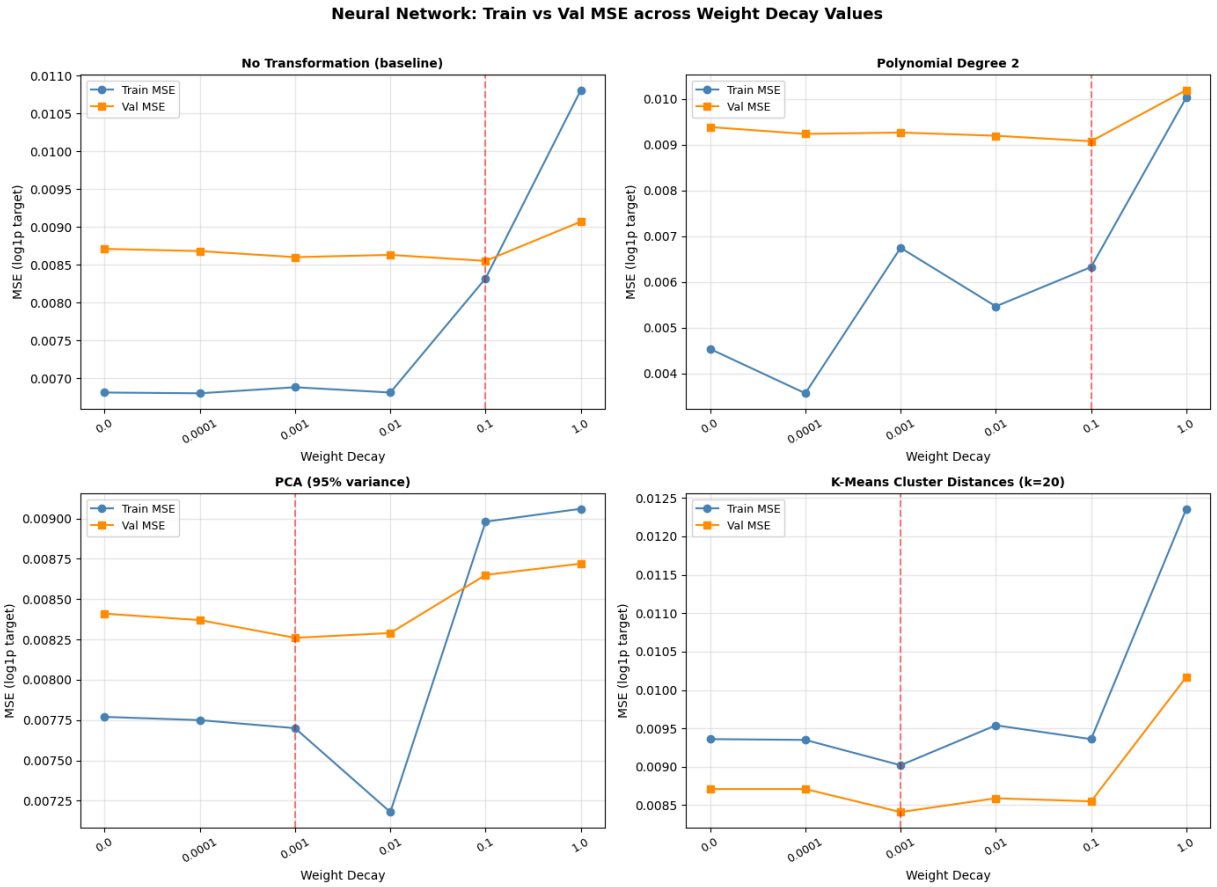


Figure 9: Neural Network - training vs. validation MSE across weight decay values for all four feature spaces.

| Model | Feature Space | k | Train MSE | Val MSE | Val $R^2$ | Observation |
|---|---|---|---|---|---|---|
| KNN | No Transformation | 1 | 0.000000 | 0.018362 | 0.315639 | Overfits heavily; validation error is high. |
| KNN | No Transformation | 3 | 0.005796 | 0.010504 | 0.608493 | Validation improves as variance decreases. |
| KNN | No Transformation | 5 | 0.007436 | 0.009598 | 0.642279 | Good bias-variance balance. |
| KNN | No Transformation | 10 | 0.009072 | 0.009363 | 0.651026 | Strong validation performance. |
| KNN | No Transformation | 20 | 0.009986 | 0.009160 | 0.658585 | Best result for the baseline features. |
| KNN | No Transformation | 50 | 0.011321 | 0.009556 | 0.643857 | Slight decline; larger k begins to underfit. |
| KNN | Polynomial Degree 2 | 1 | 0.000000 | 0.017572 | 0.345094 | Perfect train fit but weak validation. |
| KNN | Polynomial Degree 2 | 3 | 0.006365 | 0.011050 | 0.588171 | Improves from k=1 but remains weaker. |
| KNN | Polynomial Degree 2 | 5 | 0.008222 | 0.011228 | 0.581508 | Polynomial features do not help here. |
| KNN | Polynomial Degree 2 | 10 | 0.010542 | 0.011441 | 0.573592 | Higher k does not improve validation. |
| KNN | Polynomial Degree 2 | 20 | 0.013485 | 0.012701 | 0.526622 | Performance declines; underfitting appears. |
| KNN | Polynomial Degree 2 | 50 | 0.017994 | 0.015612 | 0.418122 | Worst polynomial result; too much smoothing. |
| KNN | PCA 95% Variance | 1 | 0.000000 | 0.017268 | 0.356399 | Still overfits, but slightly better than other k=1 runs. |
| KNN | PCA 95% Variance | 3 | 0.005781 | 0.011308 | 0.578530 | Error remains higher than the baseline. |
| KNN | PCA 95% Variance | 5 | 0.007429 | 0.009803 | 0.634631 | Competitive result at moderate k. |

| | | | | | | |
|---|---|---|---|---|---|---|
| KNN | PCA 95% Variance | 10 | 0.009003 | 0.009449 | 0.647839 | Good train-validation balance. |
| KNN | PCA 95% Variance | 20 | 0.010032 | 0.009054 | 0.662539 | Best overall KNN result. |
| KNN | PCA 95% Variance | 50 | 0.011400 | 0.009522 | 0.645114 | Slight decline from k=20. |
| KNN | K-Means Dist. (k=20) | 1 | 0.000000 | 0.022023 | 0.179204 | Worst k=1 result; severe overfitting. |
| KNN | K-Means Dist. (k=20) | 3 | 0.006721 | 0.013291 | 0.504655 | Improves but remains weak. |
| KNN | K-Means Dist. (k=20) | 5 | 0.008362 | 0.010614 | 0.604421 | Moderate improvement. |
| KNN | K-Means Dist. (k=20) | 10 | 0.009603 | 0.010394 | 0.612604 | Smoother predictions help slightly. |
| KNN | K-Means Dist. (k=20) | 20 | 0.010821 | 0.010023 | 0.626435 | Best K-Means distance result. |
| KNN | K-Means Dist. (k=20) | 50 | 0.012072 | 0.010577 | 0.605807 | Drops from k=20; likely underfitting. |

Table 3: KNN results.

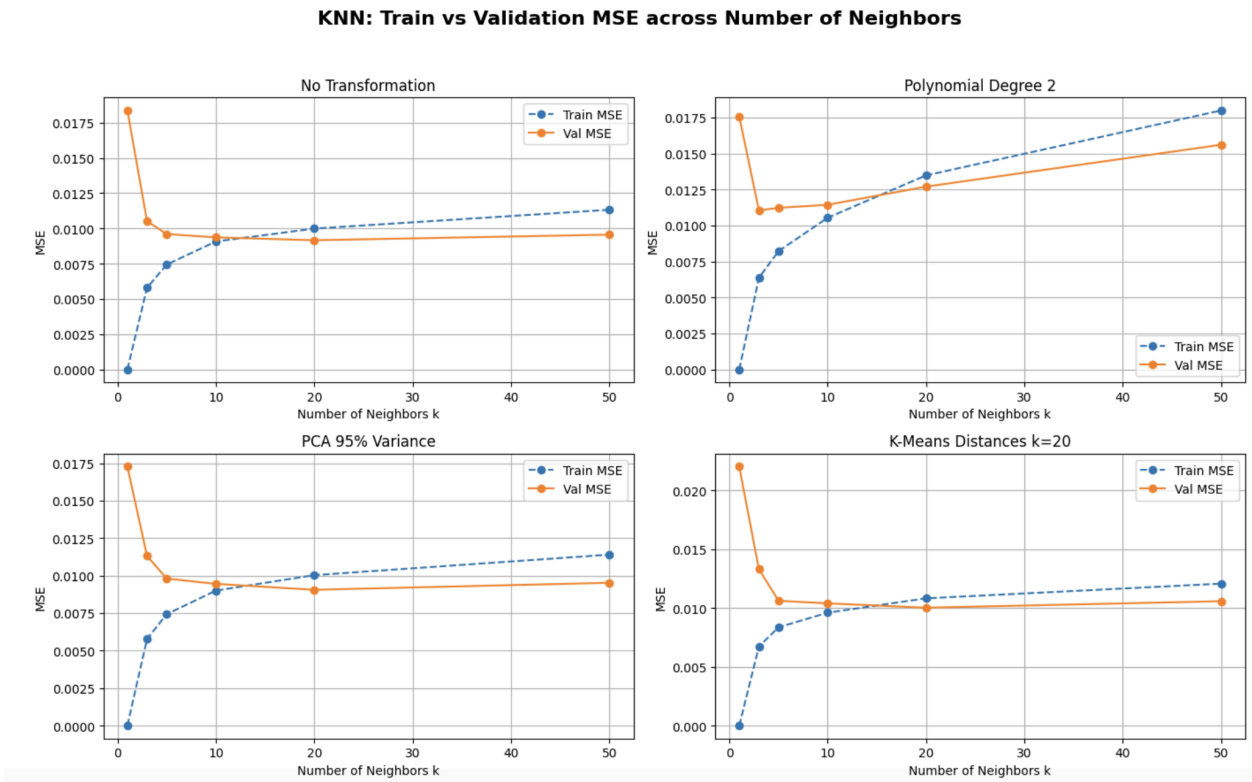

Figure 10 - KNN train MSE vs validation MSE for different feature spaces

## E. Conclusion and Analytical Discussion

**General findings.** Across all three models, the dataset's socio-economic structure was learnable to a meaningful degree. The best overall model was the Neural Network with PCA transformation and weight decay of 1e-3, achieving a validation $R^2$ of 0.692. PCA was the most consistently effective transformation across all three approaches. The key challenge throughout was managing the bias-variance tradeoff given the high dimensionality and moderate training size.

**Polynomial degree 2.** For Ridge, degree-2 features caused catastrophic overfitting at low lambda. Train MSE dropped to near 0 while val MSE was 6x higher than the baseline. Even at lambda=100, the best result was only $R^2$=0.364, far below the no-transformation baseline of 0.684. For the neural network, polynomial features caused persistent overfitting with a gap of 0.005 at all weight decay values, and the best result ($R^2$=0.662) still underperformed the baseline. For KNN, the high dimensionality of the polynomial feature space caused the curse of dimensionality and the transformation actively hurt performance. The scatter plots in Figures 4–6

did show curved relationships, but the polynomial expansion appears to have added more noise than signal, particularly when combined with the existing multicollinearity in the dataset.

**PCA (95% variance).** PCA was the best transformation across all three models. For Ridge, PCA achieved the best validation result ($R^2$=0.689) and was remarkably insensitive to the choice of lambda. Validation MSE barely changed across all six lambda values, confirming that removing multicollinearity reduced the need for explicit regularization. For the neural network, PCA with weight decay 1e-3 achieved the best result overall ($R^2$=0.692), with a very small train/val gap of 0.00056, indicating excellent generalization. For KNN, PCA produced the best KNN result ($R^2$=0.663) by reducing redundancy in the distance calculations. The consistent benefit of PCA across all three approaches confirms that the multicollinearity identified in Figure 7 was genuinely harming model performance.

**PCA(20) + polynomial degree 3 (Ridge only).** As expected, this was the most complex transformation and required the most regularization. At lambda=0.001, the validation $R^2$ was −6.2, meaning the model performed far worse than simply predicting the mean. Even at lambda=100, the best result ($R^2$=0.351) was slightly worse than the simpler Poly-2 transformation. The extra complexity added by the cubic terms did not help. The dataset does not appear to have cubic-scale nonlinearities that this transformation could capture.

**K-Means cluster distances (Neural Network and KNN).** For the neural network, the K-Means transformation performed similarly to the baseline ($R^2$=0.687 vs 0.681), confirming that community cluster membership encodes meaningful structure. The compact 20-dimensional representation was sufficient to capture most of the predictive signal. For KNN, K-Means performed better than raw features ($R^2$=0.626 vs 0.659 for PCA) but was the weakest of the four KNN feature spaces, suggesting the cluster distances did not capture the fine-grained neighborhood structure that KNN relies on as well as PCA components did.

**Effect of regularization.** For Ridge, the expected U-shape in validation MSE was clearly visible in the Poly-2 and Poly-3 panels of Figure 9. Low lambda caused severe overfitting and increasing lambda steadily improved validation performance, with the best result always at lambda=100. The baseline and PCA panels showed almost no sensitivity to lambda, since those feature spaces had minimal overfitting to begin with. For the neural network (Figure 10), the

PCA panel shows the clearest U-shape: val MSE is low at small weight decay, reaches a minimum at 1e-3, then rises as the model underfits. For KNN, the k=1 case is always severely overfit (train MSE near 0), and increasing k consistently reduces the train/val gap until around k=20, after which val MSE begins to rise from underfitting.

**Comparing models.** On shared transformations (Poly-2 and PCA), the Neural Network consistently matched or outperformed Ridge, confirming that some nonlinear structure exists in the data beyond what a linear model can capture. The gap was largest on PCA: NN achieved $R^2$=0.692 vs Ridge's 0.689. This small but consistent advantage suggests moderate nonlinearity. KNN performed competitively on the no-transform and PCA feature spaces but was hurt by the polynomial transformation in a way that Ridge and NN were not.

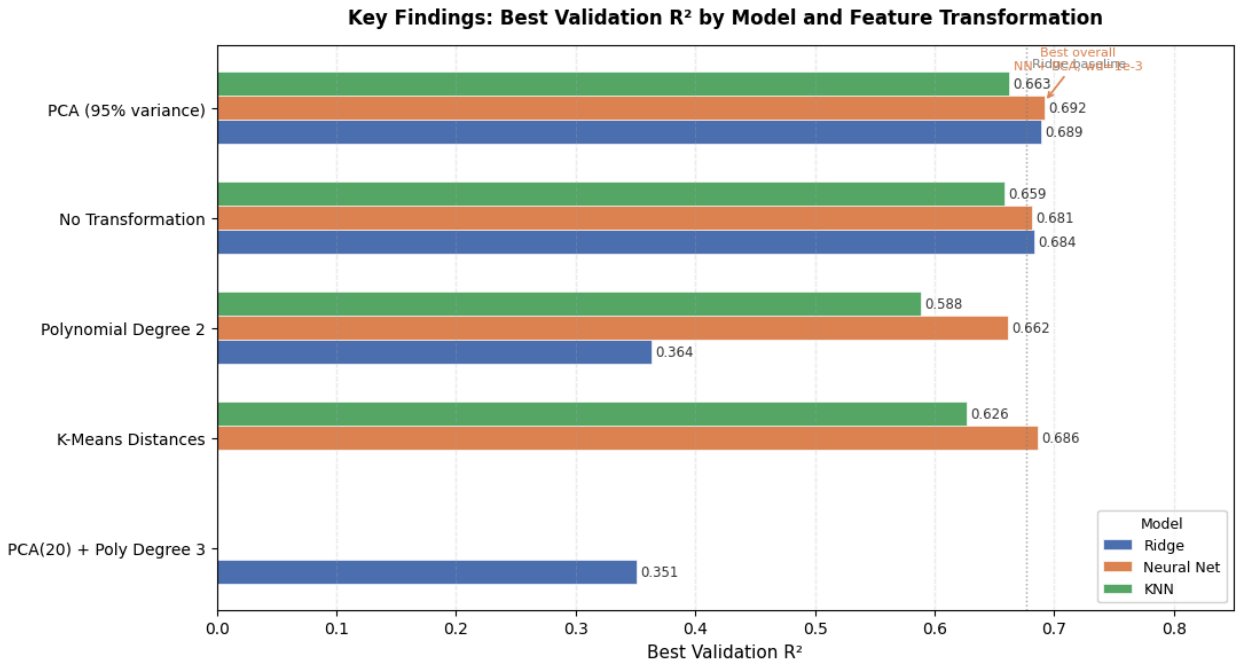


Figure 11: $R^2$ compared across all models.

**Limitations and next steps.** The log1p target transformation improved model performance but makes MSE less directly interpretable in real-world crime rate units. Future work could report mean absolute error in the original scale. The polynomial degree-3 transformation required a PCA pre-step, which may have lost information before the cubic terms were applied. An ensemble combining the predictions of the best Ridge, NN, and KNN models would be a natural

next step. Their different inductive biases could complement each other, particularly since KNN is sensitive to local structure while Ridge and NN capture global trends.